

SGT UNIVERSITY, GURUGRAM

Shree Guru Gobind Singh Tricentenary University
Department of Computer Science and Engineering
B.Tech CSE (Artificial Intelligence and Machine Learning), Section C

DATABASE MANAGEMENT SYSTEMS

TA-3 · IMPLEMENTATION

Persistent Memory Architecture for AI Agents

Implementation of a relational memory store

Submitted by

S. No.	Name	Roll number
1	Harshit Khemani	241302081
2	Kush Ahuja	241302122
3	Kushagra Agrawal	241302109
4	Madhav Bassi	241302100

Submitted to Dr. Poonam Sangwan

30 September 2026

Declaration. We declare that this report and the program in ta3-implementation/ are our own work for DBMS TA-3.

Harshit Khemani (241302081) · Kush Ahuja (241302122) · Kushagra Agrawal (241302109) · Madhav Bassi (241302100)

1. Introduction and objective

Phase 1 of this term paper reviewed persistent memory for AI agents. Phase 2 compared Notion's Lore with the same logical schema on SQLite and PostgreSQL, and added Nemori, LightMem, and *Harness the Memory*. The recommendation was a local relational store: episodic rows, semantic triples with a validity interval, a normalised entity and alias table, provenance back to the episode that supported a claim, and indexes for keyword and vector retrieval. Lore's vault stays legible, but it cannot declare those constraints. This phase implements the store.

The objective is a small program a reader can run, not a second copy of the measurement harness. It has to support write, retrieve (keyword and vector), consolidate, update, and forget, on a schema with keys and checks, and print the result of one agent session.

2. Software and tools

The memory store imports only the Python standard library. Pygments and Chrome are used to render this report. They are not imported by `demo.py`. Versions below were read on the machine that produced the transcript.

Software	Version	Role
Python	3.12.3	The language. <code>sqlite3</code> is in the standard library, so <code>python3 demo.py</code> needs no extra packages.
SQLite	3.45.1	One file, with foreign keys, CHECK, partial unique indexes, and FTS5. No server. Phase 2 is why: a relational engine can declare the constraints Lore only scans for.
sqlite3 CLI	3.45.1	Runs <code>queries.sql</code> , so the query figure is shell output.
Pygments	2.21.0	Syntax highlighting for the code figures. Not used by the demo.
Google Chrome	148.0.7778.96	Renders the figures and prints this PDF. Not used by the demo.
Linux	Ubuntu 24.04.4 LTS; kernel 6.12.94+	The machine the demo was run on.
Cursor	editor	Where the code was written. The demo does not depend on it.

PostgreSQL would add a temporal exclusion constraint, which SQLite cannot express. That is recorded under future work. SQLite is enough to show primary keys, foreign keys, checks, and a partial unique index, and it keeps the submission runnable from a clone.

3. System architecture

An agent session does not talk to SQL itself. `demo.py` calls `MemoryStore`. The class opens one SQLite connection, turns foreign keys on (SQLite leaves them off unless asked), and applies `schema.sql`. Each operation is a short transaction. Retrieval is three queries whose ranks are fused in Python with reciprocal rank fusion, the same rule Phase 2 noted in Lore, with the constant $k = 60$.

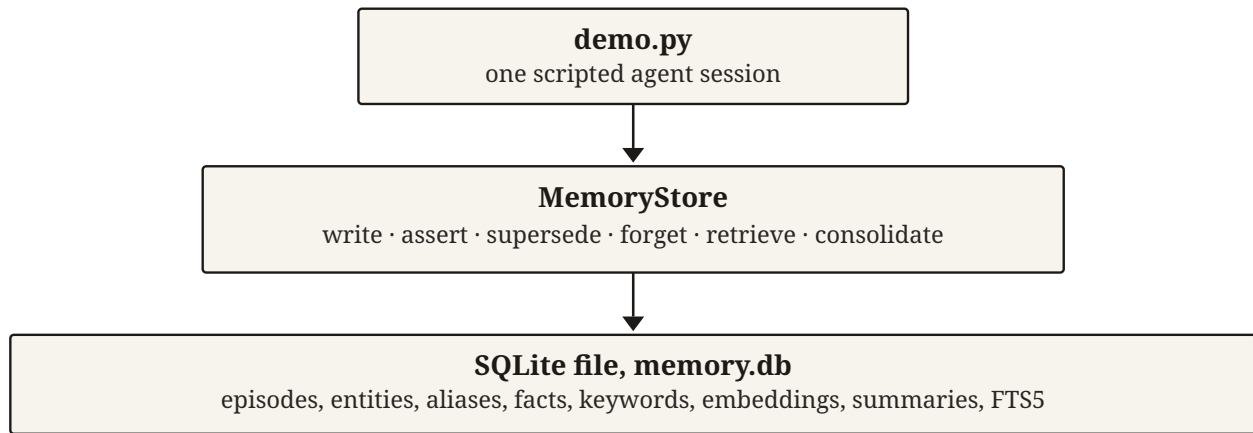


Figure 1. The session calls one module. The module is the only writer of `memory.db`.

The six memory operations from the survey in Phase 2 map onto the schema as follows. Write inserts an episode and indexes it. Assert inserts a semantic fact. Supersede is the update: the old row's interval is closed and a new row is inserted. Forget changes a status flag and keeps the row, so provenance still joins. Retrieve reads. Consolidate inserts a summary row, which is a stored result of the active facts from that session rather than a second copy of the episodes.

4. Database schema

Figure 2 is the entity-relationship diagram. `ENTITY_ALIAS.alias_key` is the primary key, so an alias belongs to one entity. That is the first-normal-form repair for a comma-joined alias cell. `KEYWORD` and `EMBEDDING` each have a check that exactly one parent foreign key is set, so those associations are real foreign keys rather than a type tag plus an integer.

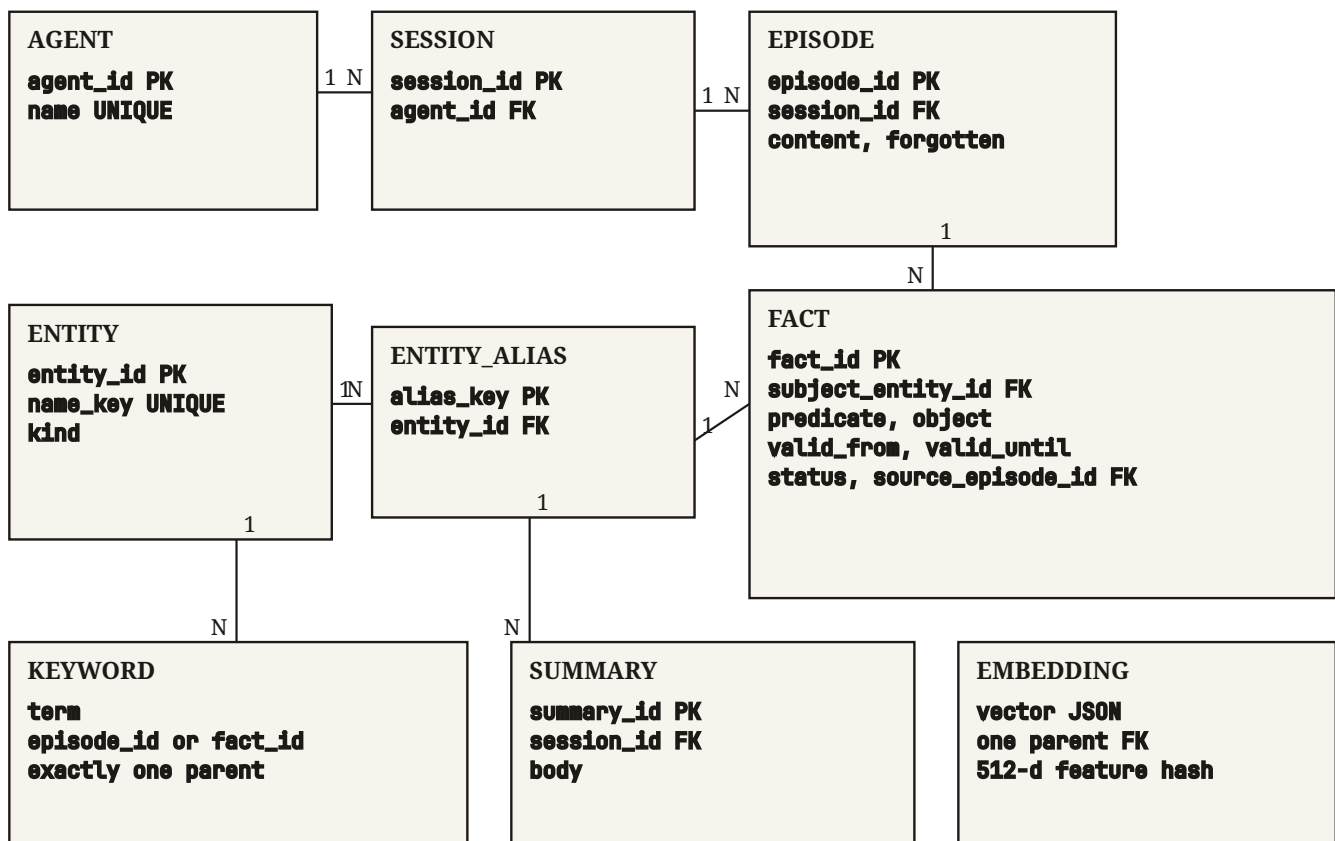


Figure 2. Crow's-foot sketch of the store. Keyword, summary, and embedding are dependent on the row they describe.

FACT is the semantic memory. `valid_from` and `valid_until` are valid time. A check requires an active fact to have an open interval, and a superseded or forgotten fact to have a closed one. The partial unique index `ux_one_active_functional_fact` allows many historical owners and at most one active owned_by, is_a, or created_by per subject. `uses` is not in that index: an application may use more than one system, and a wrong guess is forgotten rather than blocked. Views `belief_history` and `current_belief` are what the as-of query and the “who owns this now” query read.

```

58 -- Semantic memory. A fact is a triple plus when it was believed.
59 -- status = 'active' rows are the current belief. Superseding or forgetting
60 -- closes valid_until instead of deleting the row, so as-of queries still work.
61 CREATE TABLE fact (
62     fact_id          INTEGER PRIMARY KEY,
63     subject_entity_id INTEGER NOT NULL REFERENCES entity(entity_id),
64     predicate         TEXT NOT NULL
65                     CHECK (predicate IN (
66                             'is_a', 'uses', 'owned_by', 'created_by', 'depends_on'
67                         )),
68     object_text       TEXT,
69     object_entity_id  INTEGER REFERENCES entity(entity_id),
70     confidence         REAL NOT NULL CHECK (confidence > 0 AND confidence <= 1),
71     stance            TEXT NOT NULL
72                     CHECK (stance IN ('certain', 'likely', 'speculative')),
73     valid_from        TEXT NOT NULL,
74     valid_until       TEXT,
75     status            TEXT NOT NULL DEFAULT 'active'
76                     CHECK (status IN ('active', 'superseded', 'forgotten')),
77     source_episode_id INTEGER REFERENCES episode(episode_id),
78     superseded_by     INTEGER REFERENCES fact(fact_id),
79     statement         TEXT NOT NULL,
80     created_at        TEXT NOT NULL,
81     CHECK (object_text IS NOT NULL OR object_entity_id IS NOT NULL),
82     CHECK (valid_until IS NULL OR valid_from <= valid_until),
83     CHECK (
84         (status = 'active' AND valid_until IS NULL)
85         OR (status IN ('superseded', 'forgotten') AND valid_until IS NOT NULL)
86     )
87 );
88
89 -- At most one *current* object for predicates that are functions of the
90 -- subject. This is the constraint a conflict scan otherwise has to rediscover.
91 CREATE UNIQUE INDEX ux_one_active_functional_fact
92 ON fact (subject_entity_id, predicate)
93 WHERE status = 'active'
94 AND predicate IN ('owned_by', 'is_a', 'created_by');
95
96 CREATE INDEX ix_fact_source ON fact (source_episode_id);
97 CREATE INDEX ix_fact_validity ON fact (valid_from, valid_until);
98 CREATE INDEX ix_fact_object ON fact (object_entity_id);

```

Figure 3. The fact table and the partial unique index in `schema.sql`. An active fact must have an open interval. The index allows many past owners and only one current owned_by, is_a, or created_by.

5. Implementation

`memory_store.py` is the only module. Tokens are lower-cased, stopwords are dropped, and a short suffix stem (*owns* and *owned* both become *own*) makes the keyword table agree with the FTS5 porter tokenizer. Embeddings are a signed feature hash into 512 buckets, using SHA-256 so the vector does not change between processes. Cosine similarity is the dot product of two unit vectors. There is no neural model and no network call. The embedding table stores a JSON array of floats, which `sqlite3` can print.

`assert_fact` resolves names through `entity_alias`, so the session can say “the app” and hit Campus Navigator. If a functional predicate already has a different active object, the method raises `FactConflict` and does not insert. `supersede_fact` then closes the old interval and inserts the replacement in one transaction: it marks the old row superseded, inserts the new row, and sets `superseded_by`. The unique index is never asked to hold two active owners. Figure 7 is that refusal, from the run.

```
407 def retrieve(self, query: str, limit: int = 5) -> list[dict[str, object]]:
408     tokens = tokenize(query)
409     lanes = {
410         "fts": self._fts_keys(tokens),
411         "keyword": self._keyword_keys(tokens),
412         "vector": self._vector_keys(query),
413     }
414     scores: dict[str, float] = {}
415     used: dict[str, list[str]] = {}
416     for lane, keys in lanes.items():
417         for rank, key in enumerate(keys, start=1):
418             scores[key] = scores.get(key, 0.0) + 1.0 / (RRF_K + rank)
419             used.setdefault(key, []).append(lane)
420     ordered = sorted(scores, key=lambda key: (-scores[key], key))
421     hits: list[dict[str, object]] = []
422     for key in ordered[:limit]:
423         kind, raw_id = key.split(":", 1)
424         hits.append(
425             {
426                 "kind": kind,
427                 "id": int(raw_id),
428                 "score": scores[key],
429                 "lanes": "+".join(used[key]),
430                 "text": self._label(kind, int(raw_id)),
431             }
432         )
433     return hits
```

Figure 4. `retrieve` in `memory_store.py`. Three ranked lists are fused with reciprocal rank fusion, $k = 60$. The `lanes` field is what the transcript prints.

`forget_fact` sets status to forgotten and writes `valid_until`. `forget_episode` sets forgotten and deletes that episode’s keyword rows. The episode row stays, because a fact may still cite it. Deleting the episode outright is refused by the foreign key, which the demo provokes and then rolls back. `consolidate` inserts one summary row whose text is the active facts sourced from the session, with their confidences. That is compression as a materialised row. It is not a language-model summary.

demo.py is the session. The clock is a list of fixed timestamps, so a second run prints the same transcript. Figure 5 is the part that registers entities and the six turns.

```
68     store.ensure_entity(  
69         "Campus Navigator", "project", t, aliases=["the app", "navigator"]  
70     )  
71     store.ensure_entity(  
72         "DBMS project team", "team", t, aliases=["the team", "project team"]  
73     )  
74     store.ensure_entity("SQLite", "system", t, aliases=["sqlite3"])  
75     store.ensure_entity("MongoDB", "system", t, aliases=["mongo"])  
76     store.ensure_entity("AI Lab", "team", t, aliases=["the lab"])  
77     store.ensure_entity("Harshit Khemani", "person", t, aliases=["Harshit"])  
78  
79     turns = [  
80         (  
81             "2026-09-30T09:05:00",  
82             "We are building Campus Navigator, a student app for SGT University.",  
83         ),  
84         (  
85             "2026-09-30T09:10:00",  
86             "Campus Navigator is owned by the DBMS project team.",  
87         ),  
88         (  
89             "2026-09-30T09:15:00",  
90             "The app uses SQLite for persistent memory, because we need "  
91             "foreign keys and full-text search.",  
92         ),  
93         (  
94             "2026-09-30T09:20:00",  
95             "Harshit Khemani created the memory schema.",  
96         ),  
97         (  
98             "2026-09-30T09:25:00",  
99             "A teammate guessed the app uses MongoDB. "  
100             "That guess is speculative and wrong.",  
101         ),  
102         (  
103             "2026-09-30T09:40:00",  
104             "Ownership moved. The AI Lab now owns Campus Navigator.",  
105         ),  
106     ]
```

Figure 5. The scripted session in demo.py. Aliases are registered with the entity, and each turn becomes one episode.

6. Output and results

python3 demo.py deletes any existing memory.db, applies the schema, and runs the session. The figures in this section are consecutive pieces of that one transcript. Nothing in them was typed in by hand.

1. Schema

```
=====
database file: memory.db
tables: agent, embedding, entity, entity_alias, episode, episode_fts,
episode_fts_config, episode_fts_data, episode_fts_docsize, episode_fts_idx, fact,
fact_fts, fact_fts_config, fact_fts_data, fact_fts_docsize, fact_fts_idx, keyword,
session, summary
partial unique index ux_one_active_functional_fact:
CREATE UNIQUE INDEX ux_one_active_functional_fact ON fact (subject_entity_id, predicate)
WHERE status = 'active' AND predicate IN ('owned_by', 'is_a', 'created_by')
```

2. Write a session

```
=====
agent 1 Archivist, session 1
episode 1 2026-09-30T09:05:00 We are building Campus Navigator, a student app for SGT
University.
episode 2 2026-09-30T09:10:00 Campus Navigator is owned by the DBMS project team.
episode 3 2026-09-30T09:15:00 The app uses SQLite for persistent memory, because we
need foreign keys and full-text search.
episode 4 2026-09-30T09:20:00 Harshit Khemani created the memory schema.
episode 5 2026-09-30T09:25:00 A teammate guessed the app uses MongoDB. That guess is
speculative and wrong.
episode 6 2026-09-30T09:40:00 Ownership moved. The AI Lab now owns Campus Navigator.
alias 'the app' -> entity 1 Campus Navigator (project)
```

3. Assert semantic facts

```
=====
fact 1 Campus Navigator owned_by DBMS project team certain 0.90 from
2026-09-30T09:10:00
fact 2 Campus Navigator uses SQLite certain 0.95 from 2026-09-30T09:15:00
fact 3 Campus Navigator created_by Harshit Khemani certain 0.90 from
2026-09-30T09:20:00
fact 4 Campus Navigator uses MongoDB speculative 0.30 from 2026-09-30T09:25:00
=====
```

Figure 6. Schema creation and the six episodes. The partial unique index is printed from `sqlite_master`. The alias the app resolves to entity 1, Campus Navigator. The `*_fts_*` names are FTS5's own tables.

```
python3 demo.py
```

4. Update and forget

```
=====
API refused a second active owner (fact 1 is still current)
  existing: Campus Navigator owned by DBMS project team
supersede fact 1 -> fact 5 at 09:40
forget fact 4 (speculative MongoDB use) at 09:42
forget episode 5 (the wrong guess is kept in the table, hidden from retrieval)
ownership history:
  fact 1  DBMS project team  superseded  [2026-09-30T09:10:00 .. 2026-09-30T09:40:00]
  fact 5  AI Lab  active  [2026-09-30T09:40:00 .. open]
```

5. The index refuses what the API already refused

```
=====
IntegrityError: UNIQUE constraint failed: fact.subject_entity_id, fact.predicate
=====
```

Figure 7. Update and the index. The API refuses a second active owner. Supersede closes fact 1 at 09:40 and inserts fact 5. A raw INSERT of another active owned_by then fails with UNIQUE constraint failed.

```
python3 demo.py
```

6. Retrieve

```
=====
predicate lookup owned_by(Campus Navigator) = AI Lab  (fact 5, from 2026-09-30T09:40:00)
```

```
query: full-text search sqlite
```

1. episode id=3 rrf=0.0492 lanes=fts+keyword+vector
user: The app uses SQLite for persistent memory, because we need foreign keys and full-text search.
2. fact id=2 rrf=0.0484 lanes=fts+keyword+vector
Campus Navigator uses SQLite

```
query: who owns Campus Navigator
```

1. episode id=2 rrf=0.0489 lanes=fts+keyword+vector
user: Campus Navigator is owned by the DBMS project team.
2. episode id=6 rrf=0.0481 lanes=fts+keyword+vector
user: Ownership moved. The AI Lab now owns Campus Navigator.
3. fact id=5 rrf=0.0479 lanes=fts+keyword+vector
Campus Navigator owned by AI Lab
4. episode id=1 rrf=0.0469 lanes=fts+keyword+vector
user: We are building Campus Navigator, a student app for SGT University.
5. fact id=2 rrf=0.0464 lanes=fts+keyword+vector
Campus Navigator uses SQLite

```
query: MongoDB
(no hit)
```

Figure 8. Retrieval. The predicate query returns AI Lab, fact 5. Lexical fusion ranks the old ownership episode first and the current fact third. After the MongoDB rows are forgotten, the query MongoDB returns no hit.

7. Consolidate

```
=====
```

summary 1 over 5 live episodes

Consolidated beliefs from this session: Campus Navigator uses SQLite (confidence 0.95); Campus Navigator created by Harshit Khemani (confidence 0.90); Campus Navigator owned by AI Lab (confidence 0.92).

8. Provenance delete is refused

```
=====
```

DELETE episode 3 refused: FOREIGN KEY constraint failed

9. Checks

```
=====
```

PASS alias 'the app' is Campus Navigator
 PASS exactly one active owner
 PASS as-of 09:30 the owner is the DBMS project team
 PASS MongoDB fact is forgotten and not a current belief
 PASS retrieval of 'full-text search sqlite' hits a SQLite memory
 PASS forgotten MongoDB memory is not retrieved
 PASS supersede wrote a new fact id
 PASS summary names the AI Lab
 PASS PRAGMA integrity_check is ok

```
=====
```

Figure 9. Consolidation, the foreign-key refusal, and the nine checks. The summary names AI Lab and does not name MongoDB. Deleting episode 3, which fact 2 cites, is rolled back.

```
sqlite3 -header memory.db < queries.sql
```

```
== current beliefs ==
fact_id | subject | predicate | object | stance | confidence | valid_from
2 | Campus Navigator | uses | SQLite | certain | 0.95 | 2026-09-30T09:15:00
3 | Campus Navigator | created_by | Harshit Khemani | certain | 0.90 | 2026-09-30T09:20:00
5 | Campus Navigator | owned_by | AI Lab | certain | 0.92 | 2026-09-30T09:40:00

== who owned Campus Navigator at 09:30, before the hand-off ==
subject | object | status | valid_from | valid_until
Campus Navigator | DBMS project team | superseded | 2026-09-30T09:10:00 | 2026-09-30T09:40:00

== aliases (one row per alias, unique) ==
entity | alias
Campus Navigator | Campus Navigator
Campus Navigator | navigator
Campus Navigator | the app
DBMS project team | DBMS project team
DBMS project team | project team
DBMS project team | the team
SQLite | SQLite
SQLite | sqlite3
MongoDB | MongoDB
MongoDB | mongo
AI Lab | AI Lab
AI Lab | the lab
Harshit Khemani | Harshit
Harshit Khemani | Harshit Khemani

== provenance: each active fact and the episode that supports it ==
fact_id | statement | episode_id | episode
2 | Campus Navigator uses SQLite | 3 | The app uses SQLite for persistent memory, becau
3 | Campus Navigator created by Harshit Khemani | 4 | Harshit Khemani created the memory schema.
5 | Campus Navigator owned by AI Lab | 6 | Ownership moved. The AI Lab now owns Campus Navi

== foreign keys on fact ==
id | seq | table | from | to | on_update | on_delete | match
0 | 0 | fact | superseded_by | fact_id | NO ACTION | NO ACTION | NONE
1 | 0 | episode | source_episode_id | episode_id | NO ACTION | NO ACTION | NONE
2 | 0 | entity | object_entity_id | entity_id | NO ACTION | NO ACTION | NONE
3 | 0 | entity | subject_entity_id | entity_id | NO ACTION | NO ACTION | NONE

== integrity ==
integrity_check
ok
```

Figure 10. Shell queries from the same run. Three facts are current. At 09:30 the owner was the DBMS project team. Aliases are one row each. Each active fact joins to its source episode. PRAGMA integrity_check is ok.

7. Testing and observations

The demo ends in nine checks, all passed on the run that produced the figures: alias resolution, a single active owner, the 09:30 as-of result, the forgotten MongoDB fact absent from current_belief, a SQLite hit for “full-text search sqlite”, an empty result for “MongoDB”, a new fact id from supersede, the summary naming AI Lab, and PRAGMA integrity_check. The process exits 0 only if every check passes. Two consecutive runs printed the same transcript.

Two results are worth stating because they are easy to misread. First, lexical retrieval of “who owns Campus Navigator” ranks episode 2 (the DBMS project team) above fact 5 (AI Lab). The old sentence is still in the episode table, and it shares the query’s words. The question “who owns it now” is the predicate query on current_belief, which returns AI Lab only. The as-of query is

what returns the earlier owner. Keyword search and the validity interval answer different questions, and the transcript shows both. Second, forgetting removes the MongoDB rows from retrieval without deleting the entity. The entity and its alias remain, which is what a registry is for.

The foreign-key refusal in Figure 9 happens on the Python connection, which has executed `PRAGMA foreign_keys = ON`. A bare `sqlite3` shell does not turn that pragma on by itself. The keys are declared either way; enforcement is per connection. `PRAGMA foreign_key_list(fact)` in Figure 10 lists the four foreign keys whether or not the shell is enforcing them.

8. Limitations and future work

The feature hash measures token overlap. It is not a semantic embedding, and similarity is a scan of a few dozen vectors rather than an approximate index. A model can replace embed without a schema change: the column is already a vector of floats. Consolidation does not implement Nemori’s predict-calibrate loop or LightMem’s sleep-time update. It stores the active facts of one session. The partial unique index is weaker than the PostgreSQL exclusion constraint in Phase 2: it limits active rows, and it does not reject two closed intervals that overlap. The demo is a single writer, so it does not reproduce the lost-update race; the unique index is what would make that race fail instead of silently keeping both rows. There is no authentication.

Harness the Memory found that no one substrate wins every task. This program already runs a predicate query beside the fused lexical lanes. A later version can route a question to one or the other instead of always printing both. Moving the same schema to PostgreSQL would be the place to add the range exclusion constraint SQLite does not have.

9. Conclusion

TA-3 is a runnable SQLite memory for one agent. Episodes, entities, aliases, facts, keywords, embeddings, and a session summary are ordinary tables with keys and checks. The session writes six turns, refuses a second current owner until the old interval is closed, forgets a speculative fact without losing the audit trail, and answers “who owns Campus Navigator” both as a predicate query (AI Lab, from 09:40) and as a fused retrieval over the text. Nine checks pass from a clean database file.

References

1. Notion (makenotion). *lore — persistent, shared AI memory backed by Notion*. GitHub, MIT licence, commit 95c3558. <https://github.com/makenotion/lore>
2. W. Ma, J. Nan, W. Wu, and Y. Chen. *Nemori: Self-Organizing Agent Memory Inspired by Cognitive Science*. arXiv:2508.03341, 2025. Revised as *What Deserves Memory: Adaptive Memory Distillation for LLM Agents*.
3. J. Fang, X. Deng, H. Chen, N. Zhang, et al. *LightMem: Lightweight and Efficient Memory-Augmented Generation*. ICLR 2026. arXiv:2510.18866.
4. W.-C. Huang, W. Zhang, Y. Wu, Y. Chen, et al. *Harness the Memory: A Holistic Evaluation of Memory Substrates in Memory Agents*. arXiv:2608.15008, 2026.
5. Y. Du, W. Huang, D. Zheng, Z. Wang, S. Montella, M. Lapata, K.-F. Wong, and J. Z. Pan. *Rethinking Memory in AI: Taxonomy, Operations, Topics, and Future Directions*. arXiv:2505.00675, 2025.
6. C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez. *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560, 2023.

7. P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. *Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory*. arXiv:2504.19413, 2025.
 8. P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef. *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*. arXiv:2501.13956, 2025.
 9. G. V. Cormack, C. L. A. Clarke, and S. Büttcher. *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. SIGIR 2009.
 10. E. F. Codd. *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM 13(6):377–387, 1970.
 11. P. P.-S. Chen. *The Entity-Relationship Model — Toward a Unified View of Data*. ACM Transactions on Database Systems 1(1):9–36, 1976.
 12. K. Kulkarni and J.-E. Michels. *Temporal Features in SQL:2011*. ACM SIGMOD Record 41(3):34–43, 2012.
 13. K. Weinberger, A. Dasgupta, J. Langford, A. Smola, and J. Attenberg. *Feature Hashing for Large Scale Multitask Learning*. ICML 2009.
 14. C. Sciavolino, Z. Zhong, J. Lee, and D. Chen. *Simple Entity-Centric Questions Challenge Dense Retrievers*. EMNLP 2021. arXiv:2109.08535.
- Harshit Khemani, Kush Ahuja, Kushagra Agrawal, Madhav Bassi. Submitted to Dr. Poonam Sangwan, 30 September 2026.